

# Final Project: Semantic Movie Search

*Building a Vector Database with Locality-Sensitive Hashing (LSH)*

## 1. About the project

---

"Can you propose a movie that is funny, happens in space, but also a bit philosophical?" How do we transform such free text into a movie recommendation using some **semantic search**? Keep reading...

In this project, you will implement a **Locality-Sensitive Hashing (LSH) Index**, which is an example of a modern **Vector Database**—the essential data structure powering cutting-edge **AI** and **semantic search** systems. Traditional data structures fail when handling the high-dimensional data used by neural networks; LSH provides an algorithmic solution to the curse of dimensionality, allowing us to efficiently find similar vectors without performing a slow, brute-force scan.

The functional goal of this assignment is to build a high-performance semantic search engine.

Specifically, the program you implement will perform two critical steps:

1. Accept a free-text search query (a description of the movie you want).
2. Instantly return the  $k$  closest movies to your query from the provided database of approximately 5,000 film vectors, demonstrating how randomized hashing efficiently solves the nearest-neighbor search problem.

Behind the scenes, this search is purely geometric. Your free-text query is first transformed into a 384-dimensional vector, or **embedding**, using the provided pre-trained AI model (the encoder).

Similarly, each of the ~5,000 movies in the database is already represented by a similar 384-dimensional vector. Our goal is to find the  $k$  nearest (closest) vectors to your query vector, measured by **cosine similarity**. To solve this efficiently, your core task is building the LSH Index. LSH accelerates the search by using a set of **random hyperplanes** (a hyperplane is a flat boundary passing through the origin, a line in 2D, a flat plane in 3D, and a 383-dimensional flat subspace in our 384-dimensional embedding space) to partition the high-dimensional space. By checking which side of these random planes a vector falls on, we convert the vector into a short binary **hash key**, ensuring that similar vectors are *highly likely* to fall into the same 'bucket,' which dramatically reduces the number of comparisons needed.

The main data structure you will need to implement is the **LSH Index**, which uses a randomized multi-hashing approach to approximate nearest-neighbor search. This structure consists of  $L$  independent hash tables, where  $L$  equals the `num_tables` parameter. Each of these tables uses  $K$  random hyperplanes (specified by the `num_bits` parameter) to define its specific hash function. The vector is hashed into a single key by checking which side of the  $K$  hyperplanes it falls on, where a bit is set to 1 if the dot product of the vector and the hyperplane normal vector is non-negative: `bit_i = 1 if dot(vector, plane_i) >= 0 else 0`. A single hash table is often not enough — a random hyperplane may fall directly between two similar movies, assigning them to different buckets and causing one to be missed entirely. To compensate, every movie is inserted into **all**  $L$  tables. Because each table has its own independently-generated set of  $K$  random hyperplanes, the same movie receives a different hash key in each table. At query time, the query vector is hashed once per table, the matching bucket from each table is retrieved, and all candidate indices are **unioned** into a single pool. This pool is then re-ranked by precise cosine similarity to produce the final top- $k$  results. Using  $L$  independent tables dramatically increases the probability that a query and its true nearest neighbors share a bucket in at least one table.

See a cool video on LSH for more intuition: [https://youtu.be/-nMatLpm\\_0A?si=PWI9JTs8bjgO7or0](https://youtu.be/-nMatLpm_0A?si=PWI9JTs8bjgO7or0)

## 2. What You Need to Implement

You are given a database of ~5,000 movies, each represented as a 384-dimensional vector. Your job is to build a search engine that takes a plain-English query and returns the most relevant movies quickly using Locality-Sensitive Hashing. You will need to implement:

- \* Cosine\_similarity -- measures how close two vectors are (range -1 to 1).
- \* LSHIndex -- splits the vector space into buckets using random hyperplanes so you can find near-matches without scanning every movie.
- \* Search -- encodes the user query as a vector, looks up candidates in the index, and returns the top-k results.

You may build any kind of interface on top (CLI, GUI, web) -- that is up to you.

### AI Policy

AI tools are permitted for boilerplate code (file parsing, UI).

The LSH index, cosine similarity function, and search logic must be written from scratch (i.e., without using external libraries that implement them), but guidance and feedback from AI or OK.

## 3. Provided Files

You need to use the following files from <https://github.com/BGU-CSE/data-structures-26>. They are also present on the Moodle (VPL) server -- **but, remember, do not re-upload them on submission**:

| File               | What it contains                                                                                                                                                    |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| settings.json      | All config: model name, LSH parameters (num_tables, num_bits, seed), top-k                                                                                          |
| tmdb_data.json     | Movie metadata: title, overview, genres, release date, etc.                                                                                                         |
| tmdb_vectors.npy   | np.ndarray of shape (n, 384), float32, unit-normalized — one row per movie. n is the number of movies, 384 is the embedding dimension of the all-MiniLM-L6-v2 model |
| search_template.py | Starter template -- copy and rename to search.py, then fill in the TODOs                                                                                            |
| test_local.py      | Pre-submission checker -- run this locally before uploading to VPL                                                                                                  |
| requirements.txt   | List of libraries and their exact version compatible with test_local.py                                                                                             |

**IMPORTANT** - Open these files and read them before and while you read the instructions.

## 4. Your Submission: search.py

Create a file called search.py. The grader imports it as a module and calls the items below directly -- no setup function is needed. The grader loads data (model, vectors, movies) itself and passes everything your functions need as arguments.

**This is the only file that should be submitted to VPL. Use Python version 3.12 for implementation.**

**LSH Parameters**

Your search.py must use the same LSH parameters defined in settings.json, i.e.,

```
num_tables = 20  num_bits = 8  hyperplane_seed = 42
```

Therefore, you must read them from settings.json (as shown in search\_template.py).

It is recommended to test your code with different values.

**4.1 Functions and classes that you need to implement in search.py**

| Name                                              | Input Parameters                                                                                                                                                                                                                                                       | Return Value                                                                                                                                                    |
|---------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| cosine_similarity(a, b)                           | a -- np.ndarray, 1-D float32 vector<br>b -- np.ndarray, 1-D float32 vector (same shape as a)<br>Constraint: use numpy only, no sklearn / scipy                                                                                                                         | float in [-1.0, 1.0]<br>Returns 0.0 if either vector is all zeros                                                                                               |
| LSHIndex(vectors, num_tables, num_bits, **kwargs) | vectors -- np.ndarray, shape (n, d), float32, unit-normalized (movies' vectors)<br>num_tables -- int, number of independent hash tables (L)<br>num_bits -- int, number of hyperplanes per table (K)<br>**kwargs -- accept and ignore any extra arguments without error | LSHIndex object<br>(including self.planes and self.tables)                                                                                                      |
| search(query, index, encoder, movies, k=5)        | query -- str, natural-language search string<br>index -- A LSHIndex class, built over all movie vectors<br>encoder -- SentenceTransformer, loaded model<br>movies -- A list of dicts from tmdb_data.json<br>k -- int, number of results                                | A list of k movies sorted by score (high to low).<br>Each movie is a dictionary and must have at least:<br>"title" : str<br>"score" : float (cosine similarity) |

**4.2 LSHIndex class -- required attributes and methods to implement**

| Name        | Input / Description                                                                                                                                           | Type / Returns                                                                                                                                                                                                                                   |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| self.planes | Set in __init__ using a seeded RNG (Random number generator)<br>shape (num_tables, num_bits, d)<br>Stores the random hyperplane normal vectors for all tables | np.ndarray, float32<br>shape: (num_tables, num_bits, d), where d is the embedding dimension (384 for this model)                                                                                                                                 |
| self.tables | Built by _build_tables()<br>One dict per hash table<br>Each cell in a table has a list of all vectors (movies) that belong to this cell.                      | A list of num_tables dictionaries.<br>Each dictionary: {int key -> [int idx1, int idx2, ...]} key: integer hash computed from the hyperplanes. idx: position of the movie in the vectors array (0 = first movie, matching tmdb_data.json order). |

|                              |                                                                                                                                                                                     |                                                                                                                                                                                                                                                  |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>_build_tables()</code> | No arguments<br>Reads: <code>self.planes</code> shape (num_tables, num_bits, d)<br>Reads: <code>self._vectors</code> shape (n, d) (the movies' vectors)                             | A list of num_tables dicts<br>Each dict: {int key -> [int idx1, int idx2, ...]} key: integer hash computed from the hyperplanes. idx: position of the movie in the vectors array/self._vectors (0 = first movie, matching tmdb_data.json order). |
| <code>query(q_vec, k)</code> | <code>q_vec</code> -- np.ndarray, shape (d,), float32, already encoded — encoding happens in <code>search()</code> , not here<br><code>k</code> -- int, number of results to return | A list of k (float, int) tuples (score, idx) sorted descending                                                                                                                                                                                   |

**Hash key formula:**

```

bit_i = 1 if dot(vector, plane_i) >= 0 else 0
key = sum(bit_i * (2 ** i) for i in range(num_bits))

```

**4.3 VPL Upload Instructions**

1. Name your main file `search.py`.
2. Just upload `search.py` to VPL. Other files that use this file in your application should not be uploaded.
3. Do NOT upload `settings.json`, `tmdb_data.json`, `tmdb_vectors.npy`, `vpl_test.py`, or `search_template.py`.
4. `search.py` must be importable as a module without side effects. (no code runs at import time: no printing, no file loading, no model loading outside of function and class definitions)

**! VPL Library Warning -- read before submitting search.py**

The VPL server has a limited set of packages. Importing anything not listed below in `search.py` will crash your submission.

NOT available -- do not import:

sklearn, scipy, torch, tensorflow, keras, faiss, pandas

Safe to use:

numpy, sentence\_transformers (model cached), json, heapq, math, collections, itertools (Python standard library)

**4.4 Suppressing Console Warnings**

When `sentence_transformers` and `HuggingFace` load, they print several noisy messages to the console. These lines are already included in `search_template.py`. If you write `search.py` from scratch, paste it at the very top to prevent warnings.

```

import os, warnings
from transformers import logging as hf_logging

os.environ['TOKENIZERS_PARALLELISM'] = 'false'

```

```
os.environ['HF_HUB_DISABLE_SYMLINKS_WARNING'] = '1'
warnings.filterwarnings('ignore')
hf_logging.set_verbosity_error()
```

## 5. Checking your code

---

Before submitting to VPL, run the provided `test_local.py` script to catch errors early. It mirrors the three graded sections and gives immediate feedback on your implementation.

### How to run:

```
pip install -r requirements.txt
python test_local.py
```

### What it checks:

- \* Section 1: cosine\_similarity -- no data files needed
- \* Section 2: LSHIndex structure -- no data files needed
- \* Section 3: Full search recall -- requires `tmdb_data.json` and `tmdb_vectors.npy`

### Data files for Section 3:

Section 3 loads the data files whose paths are defined in `settings.json`. Make sure both files exist at those paths relative to `test_local.py`, or update `settings.json` to match your local folder layout:

```
data.json_file = tmdb_data.json
data.vectors_file = tmdb_vectors.npy
```

If either file is missing, Section 3 will print an error with the expected paths. Sections 1 and 2 results are always valid regardless.

Sections are graded independently -- a crash in one section does not affect the others.

**Good luck!**